BANGLADESH-FOCUSED

Laravel + Vue E-Commerce Step-by-Step Implementation Roadmap

From your completed Bangladesh-focused landing page to a production-ready Laravel + Vue.js store: dynamic catalog, customer account, cart, checkout, orders, payments, shipping, admin operations, CMS, analytics and advanced features.

NEXT MOVE Build Laravel commerce foundations and a dynamic catalog before advanced personalization.	YOUR STACK Laravel owns business rules and payment security; Vue.js handles interactive storefront/admin UI.
DYNAMIC CMS Landing sections, banners, collections, visibility, scheduling and deep links controlled from admin.	PRODUCTION FOCUS COD first, verified online payments, inventory safety, queues, audit logs, testing and observability.

Laravel + Vue.js mixed website edition - easy build sequence

Read This First: What Runs Where

Your completed landing page is only the visual entry point. For your current project, Laravel should own every business decision that can affect money, stock, permissions or order state. Vue.js should make the experience fast and interactive, but it should not be allowed to decide final prices, payment success, stock availability or authorization.

The previous PDF needed a framework correction.

This edition is written specifically for a Laravel backend with a mixed Blade/Vue.js website. Any Laravel/Vue-specific wording has been removed or replaced. The commerce concepts were still valid, but the implementation guidance now matches your actual stack.

Simple responsibility map

Part	Use it for	Do not put here
Laravel	Routes, validation, auth, business rules, pricing, inventory, orders, payments, refunds, policies, jobs, webhooks, reports	Do not let the browser bypass these rules.
Blade / Laravel-rendered HTML	SEO-critical page shell, product/category/home content, metadata, initial state	Do not make Blade calculate financial truth separately from services.
Vue.js	Search autocomplete, filters, variant picker, cart drawer, checkout interactions, admin forms/tables, CMS drag-and-drop	Do not store gateway secrets or trust client totals.
Database	Products, variants, stock, orders, payments, refunds, CMS, audit history	Do not overwrite historical order/payment records.
Queue / Scheduler	Email/SMS, webhook processing, reconciliation, exports, image jobs, search sync	Do not make checkout wait for non-critical work.

Recommended mixed-site approach

Keep SEO-sensitive storefront pages Laravel-rendered where practical (home, category, product, campaign), then mount Vue components for the interactive parts. The admin area can be much more Vue-heavy. If your existing app already uses a different Laravel/Vue pattern, keep it - the important rule is that Laravel remains the commerce source of truth.

Complete the Store in 12 Clear Phases

Do not build everything at once. Finish and test each phase before moving to the next. A phase is complete only when the customer flow and the admin/operations flow both work.

Phase	Build now	You are done when...
0	Freeze business rules	COD, delivery areas/fees, return window, stock policy, tax/fees, single-vendor vs marketplace are written down.
1	Laravel foundation + database	Roles, policies, status enums, core migrations, queues, storage, audit and environment setup are stable.
2	Dynamic catalog	Admin can create products/variants/stock; customer can browse category/search/product pages with real price and availability.
3	Customer auth + account	Register/login/recovery/profile/ addresses/wishlist work without blocking guest browsing.
4	Cart	Guest/account cart, login merge, quantity rules, repricing and coupon validation work server-side.
5	Checkout + COD	Customer can choose address/shipping, place one COD order, stock is reserved/updated, confirmation is shown.
6	Orders + admin fulfillment	Admin can confirm, pack, ship, deliver/cancel; customer sees order history and timeline.

Phases 7-12: Payments, Operations, CMS and Launch

Phase	Build now	You are done when...
7	Online payment	Sandbox payment is initiated by Laravel, verified server-side, duplicate callbacks are safe, reconciliation works.
8	Courier + returns/refunds	Tracking, delivery exceptions, return request, inspection and refund workflow are operational.
9	Dynamic landing CMS	Admin can reorder/schedule/publish homepage sections and campaigns without a code release.
10	Growth features	Coupons, reviews, notifications, better search, analytics and recommendations are added on reliable data.
11	Hardening + launch	Security, SEO, performance, accessibility, monitoring, backups and E2E tests pass.
12	Advanced phase	Loyalty, wallet, gift cards, marketplace/seller dashboard, B2B or mobile API only after core stability.

Your next coding target

If your landing page is already completed, start with Phase 1 and Phase 2. Do not start payment gateway work before a real order can be created safely with COD. That keeps payment bugs separate from cart, stock and order bugs.

What Every Dashboard Needs - in the Correct Order

Build dashboards as working tools around the commerce engine. Do not begin with charts. First give users and staff the actions they actually need.

Customer dashboard - build in this order

#	Module	Minimum contents
1	Account overview	Name/phone/email, recent order, pending action, address shortcut.
2	My Orders	Order list with number, date, total, payment status, fulfillment status.
3	Order Details	Item snapshot, totals, address, payment, shipment/tracking, timeline, invoice, allowed cancel/return action.
4	Addresses	Add/edit/delete/default shipping address with Bangladesh district/area fields required by your delivery rules.
5	Wishlist	Saved products/variants with current price/stock and move-to-cart.
6	Returns & Refunds	Eligible items, reason/evidence, request timeline, approved/rejected/refunded result.
7	Reviews	Review delivered purchases; show moderation/verified purchase status.
8	Notifications & Support	Order/payment updates, support tickets/contact channel.
9	Security	Password change, sessions/devices, logout other sessions, communication preferences.

Admin dashboard - build in this order

#	Module	Minimum contents
1	Staff auth + RBAC	Admin login, roles, permissions, policies, audit log.
2	Catalog	Categories, brands, products, variants, images, publish/archive, SEO fields.
3	Inventory	Warehouse/stock quantity, reservations, stock movements, low-stock alerts.
4	Orders	Operational queues, filters, detail, confirm/pack/ship/deliver/cancel, invoice.
5	Payments	Attempts, verified transaction refs, failed/pending/paid, refunds, reconciliation exceptions.
6	Shipping	Zones, rates, courier mapping, shipment/tracking, delivery exceptions.
7	Customers	Profile, order history, addresses, support context; protect sensitive data.
8	Returns/Refunds	Approve/reject, receive/inspect, restock decision, replacement/refund.
9	Promotions	Coupons, campaign dates, eligibility, limits, stacking rules.
10	Homepage CMS	Sections, banners, collections, schedule, drag order, preview, publish/rollback.
11	Reviews/Support	Moderation, verified purchase, tickets and SLA queues.
12	Reports	Sales, payment, refund, inventory, coupon, cohort/funnel reports after source data is reliable.
13	Settings	Store, payment, delivery, return policy, notification templates, feature flags.

A Practical Project Structure for Your Mixed Website

You do not need microservices. Keep one Laravel application, but separate responsibilities so controllers remain thin and business rules can be reused by storefront, admin, queues and payment webhooks.

Location	Responsibility
app/Http/Controllers/Storefront	Home, catalog, cart, checkout, account page/endpoints. Controllers validate/request orchestration, then call services/actions.
app/Http/Controllers/Admin	Admin screens/endpoints. Reuse the same commerce services and enforce policies.
app/Http/Controllers/Webhooks	Payment/courier callbacks. Keep isolated, authenticated/verified and idempotent.
app/Http/Requests	Form request validation for checkout, product admin, coupon, return/refund, etc.
app/Models	Eloquent persistence models. Keep financial/order history immutable where required.
app/Services or app/Actions	Pricing, cart, checkout, order state transitions, inventory, payment, refund, shipping and CMS resolution.
app/Policies + middleware	Object-level authorization and admin role/permission checks.
app/Jobs + Events + Listeners	SMS/email, webhook processing, invoice/export, search indexing and other async tasks.
app/Console/Commands + scheduler	Payment reconciliation, expired stock reservation release, campaign activation, data maintenance.
resources/views/storefront	Blade layouts/pages for SEO-friendly HTML and first render.
resources/js/storefront	Vue components for search, filters, variants, cart, checkout and account interactions.
resources/js/admin	Vue-heavy admin pages/components, tables, CMS drag/reorder, charts where useful.
routes/web.php / api.php / admin routes	Separate public web, async/API, admin and webhook concerns clearly.

Authentication choice

If Laravel and Vue are served from the same site/domain, Laravel session authentication + CSRF protection is usually the simplest web setup. Use Sanctum when you need a separate SPA/API client, a mobile app, or token-based first-party API access. Do not add JWT just because the frontend uses Vue.

Storefront rendering choice

For a mixed website, a practical approach is: Laravel/Blade renders the page and SEO metadata, then Vue enhances interactive areas. Example: product title/price/description are present in HTML, while Vue handles variant selection, delivery quote, quantity, wishlist and add-to-cart. The admin can be more client-rendered because SEO is irrelevant there.

Keep controllers thin

A controller should not contain 150 lines of price, stock or payment logic. Put reusable rules in services/actions and cover them with tests. The same OrderService should be called from checkout, admin and webhook recovery where appropriate.

Build COD End-to-End Before Online Payment

This is the easiest way to prove that your catalog, cart, checkout, order, stock and admin workflow are correct before a gateway adds redirects and callbacks.

Step	Laravel responsibility	Vue/customer result
1. Browse	Query only published products/active variants and current stock/price.	Customer sees real catalog.
2. Add to cart	Create/find guest or user cart; store variant + requested quantity.	Cart updates instantly.
3. Review cart	Reprice every item; validate stock, promotion, coupon and limits.	Warnings show price/stock changes.
4. Checkout	Validate contact/address; calculate shipping and final total server-side.	Customer selects address and COD.
5. Place order	DB transaction: create order snapshot + items + totals + status history; reserve/decrement stock once.	One order number is returned even if button/request retries.
6. Confirm COD	Apply COD eligibility/risk policy; mark payment method COD and order confirmed/pending-confirmation according to business rule.	Confirmation page + order appears in account.
7. Admin fulfill	Admin confirms, packs, creates shipment, ships, delivers or cancels via allowed state transitions.	Customer timeline changes.
8. Completion	Record delivered; for COD, track collection/remittance separately if courier settlement is relevant.	Order completed and becomes return/review eligible.

Important

Do not create order totals from values posted by Vue. Vue may send variant IDs, quantities, address ID, coupon code and chosen shipping/payment method. Laravel must load current prices/stock and calculate the final amount itself.

Minimum tables for this flow

products, product_variants, inventories, carts, cart_items, users (optional for guest), addresses, orders, order_items, order_addresses, order_status_history, stock_movements/reservations, payments/payment_attempts (even COD can use a payment record if you want one consistent model).

Payment Gateway: Exact Server-Side Flow

After COD works, add one online gateway in sandbox. For Bangladesh, an aggregator such as SSLCOMMERZ or shurjoPay is often the easiest first integration because one integration can expose multiple payment channels. Add direct bKash/Nagad later only when your merchant agreement and provider-issued API documentation require it.

Laravel piece	Purpose
PaymentGateway interface	Common methods such as initiate(), verify/query(), refund() where supported.
SslCommerzGateway / ShurjoPayGateway	Provider-specific request/response mapping. Credentials stay on server.
PaymentService	Creates attempt, selects provider, verifies amount/reference/status, performs idempotent state transition.
Checkout/Payment controller	Starts payment and returns safe redirect URL/session data to browser.
Webhook/IPN controller	Receives provider-to-server notification, verifies it, dispatches/executes idempotent processing.
payment_attempts + payment_events	Keep every attempt/reference/status and raw-safe metadata for audit/reconciliation.
ReconcilePayments command	Scheduled query that finds pending/mismatched transactions and checks provider status.

Exact request flow

1. Customer clicks Pay. Vue sends only checkout choices; Laravel recalculates the order.
2. Laravel creates/loads a pending-payment order and a unique payment_attempt inside a safe transaction.
3. PaymentService calls the selected gateway from the server using sandbox credentials and the exact payable amount.
4. Laravel stores provider session/payment reference and returns a redirect URL or checkout session to Vue.
5. Customer pays on the gateway-controlled page.
6. The browser return URL is only for user experience. Do NOT mark the order paid only because the browser reached success.
7. Gateway IPN/webhook (or server-side validation request after return) is verified. Laravel checks merchant transaction ID, provider reference, status, amount and currency.
8. Inside a DB transaction/lock, mark payment paid exactly once and move the order to the allowed next state. Duplicate callbacks do nothing harmful.
9. Queue the receipt/order confirmation and start fulfillment after the verified payment transaction commits.
10. A scheduled reconciliation command checks stuck pending attempts and provider/local mismatches.

Gateway order I recommend

1) COD, 2) SSLCOMMERZ OR shurjoPay sandbox, 3) production onboarding for the chosen provider, 4) direct bKash/Nagad only if needed. Never integrate four gateways before the order/payment state machine is proven.

Official provider references

SSLCOMMERZ developer docs: developer.sslcommerz.com/doc/v4/ - documents session creation, IPN and order validation. shurjoPay developer docs: web2.shurjopay.com.bd/frontend-developers/ - documents sandbox, authentication, initiate/verify and integration guidance. For direct bKash/Nagad, use the current credentials and technical documentation issued to your merchant account during onboarding.

MAKE YOUR COMPLETED LANDING PAGE DYNAMIC

Turn the Landing Page into a CMS-Driven Storefront

Do not rewrite the landing UI. Keep the components you already built and replace hardcoded arrays/text with resolved content from Laravel. Admin users should be able to change campaigns and section order without editing code.

home_sections field	Example
type	hero, category_grid, flash_sale, best_sellers, banner, collection, reviews, newsletter
title/subtitle	Customer-facing copy, optional by type
sort_order	Controls section order; Vue admin updates it by drag-and-drop
is_enabled + status	Draft/published/disabled
starts_at / ends_at	Scheduled campaign visibility
settings JSON	Layout options, image IDs, labels, limits, theme variant
source_type/source rules	Manual products, category, bestseller rule, newest, campaign, collection
deep_link / target	Product/category/search/collection/campaign destination
device visibility	Optional desktop/mobile visibility if genuinely needed
revision info	Who changed what; allow rollback for important homepage edits

Public request flow

11. Laravel HomePageService loads only sections that are published, enabled and inside the scheduled date range.
12. Each section is resolved into a typed payload. Example: flash_sale resolves real promotion dates and eligible product variants.
13. Blade loops the ordered sections and renders a matching section partial/component. Vue is mounted only where the section needs interaction such as carousel/timer/add-to-cart.
14. Unknown/disabled section types fail safely instead of breaking the whole homepage.
15. Admin Vue screen edits section settings, uploads media, previews device layouts, drags order and publishes/schedules changes.
16. Track impression/click analytics with a stable section ID so you can later measure which homepage modules convert.

Do not store product copies inside the CMS

A homepage section should reference products/collections or a rule; price, stock and sale state should still come from the catalog/promotion system. Otherwise the landing page will show stale prices or out-of-stock products.

PLAIN-LANGUAGE GLOSSARY

Terms You Will See in This Roadmap

Term	Meaning
Idempotency	If the same request/callback is sent twice, the result is still one order/payment action - not duplicates.
Webhook / IPN	The payment/courier provider server calls your Laravel URL directly to report an event. This is more trustworthy than a browser redirect alone.
Reconciliation	A scheduled check that compares your local payment record with the provider so missed callbacks can be repaired.
Stock reservation	Temporarily holding quantity for an order/payment so two customers do not buy the same last item.
Order snapshot	Copy product name/SKU/price/address/totals into the order so historical orders never change when catalog data changes.
RBAC	Role-based access control: staff roles such as Super Admin, Order Manager, Catalog Manager, Finance and Support receive only permitted actions.
Audit log	A history of sensitive admin actions: who changed status/refund/price/stock/settings, when and why.
Queue	Background work for slow/non-critical tasks such as SMS/email, exports, image processing or search sync.
State machine	Rules controlling which status can move to which next status, so impossible transitions are blocked.

Your first 30 days - simple target

Week	Target
Week 1	Business rules + DB foundation + RBAC + category/brand/product/variant/inventory migrations and admin CRUD skeleton.
Week 2	Public category/search/product pages connected to real catalog, stock and price; admin product/inventory usable.
Week 3	Customer auth/profile/addresses + guest/account cart + server

	repricing and warnings.
Week 4	Checkout + shipping calculation + COD order + admin order queue/detail/confirm/pack/ship. End the month with one complete real commerce flow.
After that	Online gateway sandbox, customer order tracking, courier, returns/refunds, dynamic landing CMS, then growth/hardening.

If you get stuck, always return to one question

Can a customer complete Browse -> Product -> Cart -> Checkout -> Order, and can an admin complete Confirm -> Pack -> Ship -> Deliver/Cancel with correct stock and totals? If not, fix that before adding more dashboards or marketing features.

01 IMPLEMENTATION SECTION

Where You Are Now - and the Correct Next Move

Your landing page is the presentation layer. The next work is to build the commerce engine and connect every landing section to real backend data.

The original blueprint deliberately prioritized a curated homepage and recommended that section order, visibility, campaign dates, product sources, banners and deep links become backend/CMS-driven. This roadmap continues from that point and turns the storefront into a full operational e-commerce system.

Do not jump straight to a beautiful admin dashboard.

First define the data model, states, business rules and API contracts. Admin screens should operate the same services and domain rules that customer checkout uses. Otherwise the project becomes UI-rich but operationally fragile.

Build order from today

17. Freeze your commerce rules: single-vendor vs marketplace, warehouses, COD policy, return window, delivery zones, taxes/fees and stock strategy.
18. Create the backend foundation: users, roles, catalog, variants, prices, inventory, carts, orders, payments, shipping, promotions, CMS and audit logs.
19. Make the catalog dynamic: category listing, search/filter, product details, variants, stock and price.
20. Implement customer authentication and account recovery, but keep guest browsing and optionally guest checkout.
21. Implement cart and server-calculated checkout totals.
22. Implement order creation and stock reservation before connecting any payment gateway.
23. Add COD first as a business flow, then online payment through a provider abstraction.
24. Add customer order history/tracking and admin order fulfillment.
25. Add shipping/courier integration and returns/refunds.
26. Connect the completed landing page to a CMS/admin module so every section is scheduled, reorderable and data-driven.
27. Add analytics, search quality, notifications, reviews, recommendations and advanced growth features only after the core path is reliable.

The production-critical customer path

Step	Customer action	System responsibility
1	Browse landing/category/search	Return only active, sellable, localized catalog data.
2	Open product	Resolve variant, price, stock, promotions, delivery estimate and return policy.
3	Add to cart	Persist cart, merge guest/account carts safely, re-price on every important transition.
4	Checkout	Validate address, shipping option, coupon, stock, totals and payment eligibility.
5	Place order	Create immutable order snapshot + items + totals; reserve/decrement stock according to policy.
6	Pay / COD	Use server-side gateway integration; never trust redirect success alone.
7	Fulfillment	Admin confirms, packs, ships, tracks, delivers or cancels.
8	After sale	Customer can view invoice, request return/refund, contact support and review after delivery.

Target Architecture - Laravel Modular Monolith + Vue.js

Start with a Laravel modular monolith. Keep catalog, cart, checkout, order, payment, shipping and CMS boundaries clear inside one application. Use Vue.js for interactive storefront/admin experiences. Split services later only if real scale or organizational needs justify it.

Recommended system shape

Layer / service	Responsibility	Notes
Laravel + Vue storefront	Home, catalog, search, cart, checkout, account	Laravel/Blade can render SEO-critical pages; Vue enhances search, filters, cart, variants and checkout. Never expose payment secrets.
Admin web (Vue-heavy)	Catalog, CMS, orders, inventory, customers, payments, reports, settings	Use Laravel policies/permissions + audit logs. Admin actions call the same domain services as checkout.
Laravel application layer	Business rules and orchestration	Single source of truth for prices, stock, coupons, orders, refunds and payment states.
Relational DB	Transactional system of record	MySQL or PostgreSQL; use transactions and constraints.
Redis/cache	Sessions/cache/rate limits/locks	Useful for carts, tokens, idempotency and hot catalog caches.
Queue/workers	Emails/SMS, webhooks, invoices, search sync, reports	Do not make checkout wait for non-critical jobs.
Object storage + CDN	Product media, banners, invoices/exports	Use resized responsive images and immutable URLs.
Search engine - later	Fast full-text, typo tolerance, facets	Add after catalog/search needs exceed DB queries.
Payment providers	COD, aggregator, direct wallet integrations	Access only through backend adapter interface.
Courier providers	Shipment create, label, tracking, COD reconciliation	Keep courier-specific code behind adapters.

Architecture principle

Pricing, discount eligibility, stock, shipping cost and final payable amount are backend-authoritative. The client may display estimates but must never be the source of truth.

Laravel + Vue module boundaries

Feature	Domain focus
auth	Laravel session auth for same-domain web; Sanctum when you expose a separate SPA/API. Login/register, OTP/password recovery, account status.
home	CMS-driven sections resolved by Laravel; Blade/Vue component renderer on storefront.
catalog	Categories, brands, products, variants, filters, pricing, stock and SEO slugs.
search	Laravel search endpoint + Vue autocomplete/filter UI; upgrade engine later if needed.
wishlist	Saved products/variants
cart	Server-backed guest/account cart; Vue cart drawer/page; totals always recalculated in Laravel.
checkout	Laravel validates address, shipping, coupon, payment eligibility and creates the order exactly once.
orders	History, detail, tracking, cancellation, return/refund
account	Profile, addresses, preferences, notifications
reviews	Verified-purchase reviews and media
support	Tickets/chat/contact entry points
admin_*	Separate admin features by operations domain, not one giant dashboard module

Foundation Rules Before Features

These decisions prevent later rewrites when payments, stock, refunds and analytics are added.

Core conventions

- Money: store monetary values in integer smallest units (paisa) or a fixed DECIMAL type. Never use floating point for financial calculations.
- Currency: BDT for launch; keep currency column anyway so the model does not assume one currency forever.
- Identifiers: use internal numeric/UUID IDs plus human-friendly order numbers such as EC-260828-000123.
- Time: store timestamps in UTC; render in Asia/Dhaka for users and operations.
- Phone: normalize Bangladesh numbers to one canonical format such as +8801XXXXXXXXX; preserve display formatting separately if needed.
- Soft delete carefully: products/users may be archived, but financial/order/payment records should remain immutable and auditable.
- Status history: every significant order/payment/fulfillment state transition should have who/when/from/to/reason.
- Idempotency: place-order, payment callback, webhook processing, refund and courier create operations must safely tolerate retries.
- Snapshots: copy product name/SKU/unit price/discount/tax into order items so old orders do not change when the catalog changes.

Separate statuses instead of one overloaded order status

Dimension	Example values	Why
Order	pending, confirmed, completed, cancelled	Customer/business lifecycle.
Payment	unpaid, pending, paid, partially_refunded, refunded, failed	Independent from fulfillment.
Fulfillment	unfulfilled, processing, packed, shipped, delivered, returned	Warehouse/courier progress.
Return	none, requested, approved, rejected, received, resolved	After-sales workflow.

Important

A paid order can still be unfulfilled. A delivered order can later become partially refunded. Separate states make real operations possible.

Database Blueprint

Create the tables early, but keep optional advanced modules behind feature flags or later migrations.

Domain	Recommended tables
Identity	users, user_profiles, addresses, roles, permissions, user_roles, sessions/device_tokens
Catalog	categories, brands, products, product_variants, product_images, attributes, attribute_values, variant_attribute_values
Pricing	price_lists or variant_prices, scheduled_prices, tax_classes (optional)
Inventory	warehouses, inventories, stock_movements, stock_reservations
Shopping	carts, cart_items, wishlists, wishlist_items, recently_viewed
Orders	orders, order_items, order_addresses, order_status_history, order_notes
Payments	payments, payment_attempts, payment_events, refunds, reconciliation_records
Shipping	shipping_zones, shipping_methods, shipping_rates, shipments, shipment_events
Promotions	promotions, promotion_rules, promotion_actions, coupons, coupon_usages
CMS	announcements, banners, homepage_sections, collections, collection_products, cms_pages, menus
Engagement	reviews, review_media, questions, notifications, newsletter_subscribers
Support	support_tickets, ticket_messages, return_requests, return_items
Operations	settings, audit_logs, exports, import_jobs, webhook_logs, integration_credentials metadata

Product model - minimum fields

Entity	Must-have fields
Product	id, slug, name, description, brand_id, category_id, status, seo fields, published_at
Variant	id, product_id, sku, barcode, option summary, weight/dimensions, active

Entity	Must-have fields
Price	variant_id, regular_price, sale_price, sale_start/end, currency
Inventory	variant_id, warehouse_id, on_hand, reserved, available
Image	product/variant relation, url, alt_text, sort_order, primary
Category	parent_id, name, slug, image, status, sort_order, seo fields

Order financial snapshot

Field group	Examples
Items	subtotal_before_discount, item_discount_total
Order discounts	coupon_discount, promotion_discount
Shipping	shipping_fee, shipping_discount
Tax/fees	tax_total, payment_fee if legally/business-supported
Grand total	total_due, total_paid, total_refunded
Currency	BDT

Dynamic Catalog: Your First Customer-Facing Feature After the Landing

Before login, build browsing quality. A customer should be able to discover and understand a product without an account.

Category / listing page

Area	Required behavior
Navigation	Breadcrumbs, category tree, subcategories, brand shortcuts.
Filters	Price range, brand, rating, availability and category-specific attributes such as size/color/storage.
Sorting	Relevance/default, newest, price low-high/high-low, popularity; only expose real signals.
Pagination	Cursor or page-based API; preserve filter state; lazy load without losing scroll position.
Cards	Primary image, name, price, discount, rating count, stock/availability and wishlist.
SEO/web	Canonical URLs, clean slugs, metadata, indexable category pages, structured data where appropriate.

Product details page

- Image gallery with variant-aware images.
- Product title, brand/seller identity, SKU where useful, rating/review count.
- Regular price, promotional price, discount math and campaign end time only when real.
- Variant selection that immediately updates price, image, stock, SKU and delivery eligibility.
- Delivery location selector, estimated fee/ETA, COD availability and return policy before checkout where possible.
- Quantity guard using available inventory and per-order limits.
- Specifications/description, warranty/authenticity claims backed by real data.
- Related products, frequently bought together later, recently viewed.

Catalog API minimum

Method	Endpoint	Purpose
GET	/categories	Navigation/category tree
GET	/products	Listing/search filters/pagination
GET	/products/{slug}	Product + variants + media + availability
GET	/brands	Brand discovery
GET	/collections/{slug}	Campaign/editorial collection

Method	Endpoint	Purpose
GET	/search/suggestions?q=	Autocomplete, categories, products, recent/popular context

Customer Authentication and Account

Use phone-first UX for Bangladesh where appropriate, but design identity so email and social login can coexist later.

Recommended authentication flows

Flow	Steps	Security
Register by phone	Phone -> OTP -> name -> optional password/email	OTP expiry, attempt limit, resend cooldown, rate limit by phone/IP/device.
Password login	Phone/email + password -> session	Hash password with modern algorithm; throttle failed attempts.
OTP login	Phone -> OTP -> session	Useful for low-friction return users; protect against abuse/cost spikes.
Forgot password	Verify phone/email -> reset token/OTP -> new password	Invalidate old reset token and optionally old sessions.
Logout	Revoke current session/token	Support logout-all-devices separately.
Account security	View active devices/sessions, change password, verify email	Notify user about sensitive changes.

Guest strategy

Do not force login to browse or add to cart. If the business allows, guest checkout generally reduces friction. You can offer account creation after purchase, or link an order to an account when the same verified phone is used.

Privacy rule

Collect only data needed for the order and customer relationship. Never store gateway PINs, OTPs, card numbers or wallet secrets.

Customer profile modules

- Profile and verified phone/email
- Address book with default shipping/billing
- Communication preferences
- Saved wishlist
- Order history and tracking
- Return/refund requests
- Reviews and ratings
- Support tickets
- Notification center
- Security/session management

Cart Engine

Cart looks simple but it is where stale price, stock and promotion bugs begin. Revalidate aggressively.

Cart rules

- Guest cart gets a stable anonymous cart ID; authenticated users have an account cart.
- On login, merge guest and account carts by variant, applying quantity limits and current availability.
- Store requested quantity, not final financial truth. Recalculate prices, promotion eligibility and stock before display/checkout.
- Cart item should reference variant ID, not only product ID.
- Return structured warnings: price_changed, promotion_ended, insufficient_stock, variant_unavailable.
- Do not silently remove unavailable items; move them to an unavailable section and explain why.
- Coupon application must be server-side and return detailed eligibility/rejection reason.

Cart API

Method	Endpoint	Notes
GET	/cart	Current canonical cart + totals/warnings
POST	/cart/items	Add variant + quantity
PATCH	/cart/items/{id}	Change quantity
DELETE	/cart/items/{id}	Remove item
POST	/cart/coupon	Apply coupon code
DELETE	/cart/coupon	Remove coupon
POST	/cart/merge	Guest -> authenticated merge

Checkout

Checkout is a server-orchestrated workflow: address -> shipping -> coupon -> payment option -> final validation -> order.

Recommended checkout steps

28. Identify customer: signed-in or guest contact with verified/valid phone.
29. Select or create delivery address, including district/area/postcode where required by shipping logic.
30. Calculate available shipping methods, fee and ETA from cart weight/value/location.
31. Revalidate coupon/promotions and show exact discount breakdown.
32. Display available payment methods based on order amount, location, fraud rules and provider status.
33. Show final review: items, quantities, prices, discounts, shipping, total, policies and contact details.
34. Create the order once using an idempotency key. Do not create duplicate orders on double-tap or browser retry.
35. If online payment: create a payment attempt and redirect/open provider checkout. If COD: confirm COD workflow.

Checkout validation matrix

Validation	When	Failure behavior
Variant active	Before order	Block affected line and refresh cart.
Stock available	Before order and, if needed, before fulfillment	Return exact shortage; no negative stock.
Price current	Before order	Recalculate and require customer confirmation if total changed.
Coupon eligible	Before order	Remove/reject with reason; recalculate.
Shipping service	Before order	Offer valid alternative or block unsupported address.
Payment method	Before payment creation	Hide/disable unsupported gateway or COD rule.
Order total > 0	Before order/payment	Block invalid financial state.

Stock strategy

For scarce products, reserve stock for a short TTL when the order is created and release it when payment expires/cancels. For normal stock, you may decrement on confirmed order. Whatever policy you choose must be explicit and transactional.

Order Lifecycle and Fulfillment

Model the full lifecycle now so customer and admin dashboards use the same states and actions.

Suggested lifecycle

Stage	Order	Payment	Fulfillment	Allowed actions
Created - COD	pending/confirmed	unpaid	unfulfilled	Admin/customer cancellation per rule
Created - online	pending	pending	unfulfilled	Payment retry/expire/cancel
Payment confirmed	confirmed	paid	unfulfilled	Start fulfillment
Warehouse processing	confirmed	paid/unpaid COD	processing/packed	Pack, invoice, assign courier
In transit	confirmed	paid/unpaid COD	shipped	Track, delivery exception
Delivered	completed	paid or COD collected	delivered	Review, return request
Cancelled	cancelled	unpaid/refunded	unfulfilled/cancelled	Restock/release reservation
Returned/refunded	completed/cancelled per policy	partial/full refund	returned	Close after inventory/payment resolution

Order detail must preserve

- Order number and timestamps
- Contact and address snapshot
- Item/SKU/variant snapshot
- Price and discount breakdown
- Shipping method/fee/ETA snapshot
- Payment method and verified transaction reference
- Status timeline
- Shipment tracking
- Invoice/receipt
- Cancellation/return/refund history
- Support notes where customer-visible

Cancellation policy engine

Do not implement cancellation as a generic delete. Define when a customer may cancel, whether payment is captured, whether the parcel has shipped, what restocking/refund action is required, and who approved the change.

Payment Architecture for Bangladesh

Launch with an abstraction so COD, an aggregator and direct wallet integrations can coexist without rewriting checkout.

Payment provider contract

Operation	Purpose
isAvailable(context)	Decide whether method can be shown for this order/customer.
createPayment(order)	Create provider session/payment and store local attempt first.
handleReturn(request)	Handle browser/app redirect, but do not trust it as final proof.
handleWebhook/IPN(request)	Verify authenticity, deduplicate and update payment state transactionally.
query(providerRef)	Server-to-server status confirmation/recovery.
refund(payment, amount)	Full/partial refund when provider supports it.
reconcile(date/range)	Compare provider settlement/transactions with local records.

Recommended launch order

P0

Cash on Delivery

Implement the business rules first: eligibility, confirmation, delivery/return risk, COD amount, courier remittance tracking.

Deliverables: COD payment record | eligibility rules | admin confirm/cancel | collection status

P0/P1

Aggregator gateway

Use one gateway integration to unlock multiple online payment channels with a single backend flow.

Deliverables: sandbox | initiation | IPN/webhook | server validation | refund/reconciliation

P1

Direct bKash

Add when you have direct merchant credentials and a commercial reason for tighter wallet UX/control.

Deliverables: provider adapter | token management | create/execute/query | callback verification

P1/P2	Direct Nagad or another provider
	Integrate only against the current merchant documentation/credentials provided during onboarding. Deliverables: provider adapter | sandbox/live config | verification | reconciliation

Current provider implementation notes

Provider	Implementation pattern	Roadmap guidance
SSLCOMMERZ	Backend creates a session; provider sends IPN; merchant validates transaction server-to-server; hosted or easy checkout options.	Good launch aggregator when it fits your commercial agreement. Always validate amount/status before marking paid.
shurjoPay	REST flow with authentication token, initiate payment and verify transaction; sandbox is documented.	Alternative aggregator/provider; same local payment state machine and webhook discipline.
bKash PGW	Tokenized/checkout products exist; use merchant credentials and current developer docs for exact API/version.	Keep all secrets/server calls in backend. Verify/query before fulfillment.
Nagad	Merchant onboarding is publicly available; exact technical integration may depend on merchant onboarding/current docs.	Do not hardcode third-party blog endpoints. Prefer provider-issued docs or an approved aggregator integration.

Payment correctness rule

A success redirect is UX, not accounting truth. Mark an order paid only after authenticated server-side notification and/or provider validation/query confirms the expected transaction, currency and amount.

Payment attempt table - recommended fields

Field	Examples
identity	id, order_id, provider, method, attempt_no
money	amount, currency
references	merchant_transaction_id, provider_payment_id, provider_transaction_id
state	created, pending, paid, failed, cancelled, expired, refunded
security	idempotency_key, verified_at, signature/hash verification result
raw audit	sanitized request/response references, webhook_event_id; never sensitive credentials/PIN/card data

Online Payment - Exact End-to-End Process

This flow should be identical conceptually across gateways even if provider APIs differ.

36. Customer confirms checkout. Backend revalidates cart, shipping, discounts and order total.
37. Backend creates order in pending state and creates payment_attempt with a unique merchant transaction/invoice ID.
38. Backend calls provider create/initiate endpoint with server credentials, amount, currency and callback/IPN URLs.
39. Backend stores provider session/payment reference and returns only the safe redirect/checkout information to the client.
40. Customer completes payment on provider-controlled UI.
41. Provider redirects the customer and separately sends an IPN/webhook when supported.
42. Backend verifies notification authenticity and calls provider validation/query where required. It verifies transaction ID, merchant reference, status, amount and currency.
43. Inside a DB transaction/lock, backend changes payment from pending to paid exactly once and moves order to confirmed. Duplicate notifications become no-ops.
44. Queue sends confirmation SMS/email/push and triggers fulfillment. The user sees an order confirmation page sourced from backend order/payment status.
45. A reconciliation job later detects cases where provider is paid but local status was missed, or vice versa, and sends operations alerts.

Failure and retry handling

Case	Correct behavior
User cancels provider page	Keep order/payment cancelled or pending-expiry according to policy; allow safe retry with a new payment attempt.
Network drops after payment	User can reopen order; backend queries provider/reconciliation rather than forcing another payment.
Duplicate callback/IPN	Deduplicate by provider event/transaction ID and local state lock.
Amount mismatch	Never mark paid; flag for manual review/security alert.
Payment succeeds after order expiry	Query/validate, then either revive order if stock/rules allow or refund/manual-review.
Refund requested	Create local refund record -> call provider -> wait/verify -> update refunded totals -> notify customer.

Secrets and environments

- Separate sandbox and production credentials/config.
- Store secrets in server environment/secret manager, never Git, Blade templates, or browser JavaScript bundles.
- Use HTTPS for all callbacks and admin/storefront traffic.
- Log provider references and status changes, but redact tokens, credentials and sensitive payload fields.
- Rotate credentials and revoke old ones when staff/vendors change.

Delivery, Courier and Fulfillment

Customers should know delivery fee and a reasonable ETA before checkout; operations should have a deterministic shipment workflow.

Shipping configuration

Module	What admin controls
Zones	Country -> division/district/city/area groupings; remote-area rules.
Methods	Standard, express, pickup, seller-specific later.
Rates	Flat, weight-based, order-value-based, zone-based, free-shipping threshold.
COD rules	Eligible locations, max order value, extra verification, customer risk flags.
ETA	Handling days + courier service estimate + cutoff/holiday logic.
Packaging	Weight/dimensions, parcel count, fragile/oversize flags.

Courier adapter flow

46. Admin/worker packs order and confirms parcel data.
47. Backend creates shipment with courier adapter; persist courier consignment/tracking ID.
48. Print/download label if courier supports it.
49. Courier webhook/poll updates shipment events.
50. Map courier-specific statuses into your canonical shipment statuses.
51. On delivered COD, record collected/remitted state separately from delivery state.
52. On failed/returned shipment, update inventory/payment/COD risk according to policy.

Do not tie order status directly to one courier string.

Keep a canonical shipment state and save the original courier event for audit. This lets you switch or use multiple couriers.

Customer Dashboard - Complete Module Plan

Keep it compact and task-focused. Customers open the dashboard to manage orders, addresses, refunds and account security - not to read analytics.

Page	Must show	Primary actions
Overview	Recent orders, active delivery, wishlist count, support/refund alerts	Track order, continue shopping, manage profile
My Orders	Filter by recent/processing/shipped/delivered/cancelled	Open detail, cancel when allowed, reorder
Order Detail	Timeline, items, totals, payment, shipment, invoice	Track, retry payment, cancel, request return, contact support
Returns & Refunds	Request list, statuses, expected refund/credit	Create request, upload evidence, view resolution
Wishlist	Saved products with price/stock status	Move to cart, remove, share later
Addresses	Saved delivery addresses	Add/edit/delete/set default
Profile	Name, phone, email, preferences	Edit/verify fields
Reviews	Eligible purchases + submitted reviews	Write/edit within policy, add media
Notifications	Order/payment/promotion messages	Mark read, manage preferences
Support	Tickets/conversations	Open ticket, attach files, reply
Security	Password, sessions/devices	Change password, logout sessions

Customer dashboard UX rules

- Mobile-first: bottom navigation or compact account menu; no giant desktop-style sidebar on small screens.
- Order timeline should use customer language such as Confirmed, Packed, Shipped, Delivered.
- Every error state must include what happened and the next available action.
- Never expose internal fraud/admin notes or provider secrets.
- Use skeletons for initial loads and optimistic UI only for reversible low-risk actions such as wishlist.

Admin Dashboard Foundation

Admin is an operations product. Optimize for speed, correctness, permissions, bulk actions and auditability.

Roles

Role	Typical access
Super Admin	All modules, role management, integration settings, sensitive exports.
Operations	Orders, shipments, returns, customer service; limited settings.
Catalog Manager	Products, categories, brands, collections, media, SEO.
Inventory Manager	Warehouses, stock, adjustments, purchase/receiving if implemented.
Marketing	Promotions, coupons, homepage CMS, banners, newsletter/audience tools.
Finance	Payments, refunds, settlements, COD reconciliation, financial reports.
Support Agent	Customers, orders read access, tickets, approved operational actions.
Analyst	Reports/analytics read-only; no mutations.

Admin shell

- Global search for order number, phone, email, SKU, payment transaction and tracking ID.
- Persistent filters and saved views for operations queues.
- Permission-aware navigation and action buttons.
- Audit badge/history on sensitive changes.
- Bulk actions with explicit confirmation and background jobs for large exports/imports.
- Notification center for failed payments/webhooks, low stock, shipment exceptions and reconciliation issues.

Security

Admin authentication should be stricter than customer auth: MFA/OTP where possible, shorter sessions for sensitive roles, IP/device telemetry, rate limiting and immutable audit logs.

Admin Dashboard - Overview and Operational Queues

The first admin page should answer: what needs attention now?

Top KPIs

Metric	Definition
Gross sales	Paid/completed order value within selected period; define exclusions explicitly.
Orders	Placed/confirmed/completed counts and trend.
Average order value	Revenue / qualifying orders.
Payment success rate	Successful online attempts / initiated online attempts.
COD share	COD order/GMV share and delivery success later.
Refunds	Count and amount; full vs partial.
Fulfillment backlog	Confirmed orders not yet shipped beyond SLA.
Low stock	Variants below threshold, sorted by sales velocity later.

Action queues

- Orders needing confirmation
- Paid orders waiting for fulfillment
- Orders approaching shipping SLA
- Payment mismatches/reconciliation exceptions
- COD orders needing verification
- Return/refund requests awaiting review
- Courier delivery exceptions/returns
- Low/out-of-stock variants
- Failed CMS campaign assets/expired campaigns
- Support tickets near SLA breach

Use simple cards plus tables. Avoid vanity charts that do not drive an action.

Admin: Orders and Fulfillment

This is the most operationally important admin module after catalog.

Order list

- Search by order number, customer phone/email, transaction ID, tracking ID.
- Filter by date, order/payment/fulfillment/return status, payment method, district/zone, courier, amount.
- Saved views: New COD, Paid-Unfulfilled, Ready to Ship, Delivery Exception, Refund Pending.
- Bulk print invoices/labels only for eligible selected orders.

Order detail panels

Panel	Content / actions
Summary	Order/payment/fulfillment statuses, risk flags, totals, timestamps
Customer	Contact + account history summary; privacy-safe
Items	SKU, variant, qty, price snapshot, stock/fulfillment status
Payment	Attempts, provider refs, verified status, refund controls by permission
Shipment	Courier, tracking, label, shipment events, delivery exception
Timeline	All state transitions and automated/system events
Notes	Internal notes separated from customer-visible messages
Actions	Confirm, cancel, pack, ship, refund/return according to current state and permissions

Guardrail

Admin buttons must call domain actions such as confirmOrder() or approveRefund(), not directly patch arbitrary status fields. Every action validates current state and writes history.

Admin: Catalog, Inventory and Pricing

Catalog editing should be safe enough for non-developers and strong enough for thousands of variants.

Product editor tabs

Tab	Fields / behavior
Basic	Name, slug, brand, category, status, short/long description, tags
Media	Drag/reorder images, alt text, variant-specific images, image validation
Options/Variants	Size/color/storage attributes, SKU/barcode, active state, dimensions/weight
Pricing	Regular/sale price, schedule, cost optional, minimum price guard
Inventory	Warehouse stock, low-stock threshold, stock policy, adjustments
Shipping	Weight, dimensions, shipping class, fragile/oversize
SEO	Title, description, canonical options, social image
Related	Collections, upsell/cross-sell/manual recommendations
History	Who changed what and when

Inventory rules

- Every stock adjustment creates stock_movement with reason, quantity delta, user/system source and reference.
- Available stock = on_hand - reserved, computed consistently.
- Order cancel/payment expiry releases reservation exactly once.
- Return restock requires inspection policy; do not automatically restock damaged returns.
- Use low-stock alerts and prevent negative stock unless overselling is an explicit product policy.

Bulk operations

- CSV import with validation preview and row-level errors
- Bulk price update with percentage/fixed change and dry run
- Bulk publish/archive
- Bulk category/brand assignment
- Inventory import per warehouse
- Media migration only via background job

Admin: Customers, Support, Returns and Reviews

Give staff enough context to solve problems without exposing or editing sensitive fields unnecessarily.

Customer profile

Area	Admin view/action
Identity	Name, verified contacts, account status; mask where role requires.
Orders	Lifetime orders, recent orders, cancellations, delivery success.
Addresses	View for support/operations; controlled edit policy.
Support	Tickets, contact history, internal notes.
Returns/refunds	History and open requests.
Risk	Operational flags such as repeated COD refusal; manual notes with audit trail.
Marketing consent	Current opt-in/out state; never force opt-in.

Return request workflow

53. Customer chooses delivered order/item and eligible return reason.
54. System checks return window, item/category exclusions and previous quantities.
55. Customer submits quantity, reason and optional evidence.
56. Admin approves/rejects with reason; create reverse shipment/drop-off instruction if needed.
57. Warehouse receives and inspects item; choose restock/non-restock disposition.
58. Create refund or replacement resolution.
59. Update refund/payment/order totals and notify customer.

Review moderation

- Only mark Verified Purchase when tied to delivered order item.
- Moderate abuse/spam, not negative sentiment.
- Allow admin response where useful.
- Track review status and moderation reason.
- Aggregate rating only from eligible published reviews.

Make the Landing Page Fully Dynamic from Admin

This directly implements the architecture recommendation from your first blueprint: component-driven sections, not hardcoded homepage content.

Homepage section model

Field	Example
key/type	hero, category_grid, flash_sale, collection, product_grid, banner, brands, budget, review, trust_strip
title/subtitle	Localized display copy
sort_order	Drag/reorder value
is_visible	Master visibility toggle
starts_at/ends_at	Campaign scheduling
audience	all, guest, authenticated, new_customer, returning; advanced later
device	all/web/mobile if genuinely needed
source_type	manual_products, category, collection, bestseller_query, newest_query, campaign
source_config	JSON/query parameters constrained by section schema
cta	label + deep link/route
design_variant	Approved design-system variants only - not arbitrary CSS

Admin CMS features

- Drag-and-drop section reorder with optimistic UI + server sort order.
- Live preview for desktop/tablet/mobile before publishing.
- Draft/published states and scheduled publish windows.
- Banner/media library with aspect-ratio validation and alt text.
- Deep-link builder to category/product/collection/search or external campaign URL.
- Collection builder: manual products or rule-based source.
- Flash sale builder with real start/end, stock and promotion linkage.
- Per-section analytics: impressions, clicks, CTR and downstream conversion later.
- Revision history and rollback for homepage changes.

Design-system rule

Make content dynamic, not layout chaotic. Admin should choose approved component types and variants; do not let arbitrary margins/colors/fonts create an unmaintainable landing page.

Homepage API response concept

GET /home returns an ordered array of resolved sections. Each section contains a stable type, metadata, analytics key and typed payload. The frontend maps type -> Vue/Blade component. Unknown types are ignored safely so Laravel and frontend component versions can coexist.

Promotions, Coupons and Campaign Engine

Do not encode every campaign as an if-statement. Build a small rule/action engine.

Promotion building blocks

Type	Examples
Eligibility	Date window, customer segment, channel, min basket, first order, payment method, category/brand/product
Benefit	Fixed discount, percentage discount, free shipping, buy X get Y, product-level discount
Limits	Global usage, per-customer usage, budget/cap, max discount, quantity limit
Priority	Stackable vs exclusive, priority order, best-discount strategy
Coupon	Code-required promotion with normalized case and brute-force protection for private codes

Calculation pipeline

60. Load current prices.
61. Evaluate item-level promotions.
62. Evaluate basket/order-level promotions.
63. Apply coupon eligibility and stacking rules.
64. Calculate shipping and shipping promotions.
65. Calculate tax/fees if applicable.
66. Return transparent breakdown and final total.
67. Re-run same pipeline at order creation to prevent client tampering.

Campaign integrity

Discount percentage, countdown, “official”, “free delivery” and stock urgency shown on the landing page must come from the same real pricing/inventory/promotion rules used at checkout.

Search, Recommendations and Personalization

Launch search quality before AI. Good autocomplete, filters and ranking usually matter more than a complex recommendation model.

Search maturity path

Level	Capabilities
V1 - DB search	Name/SKU/category/brand search, filter/sort, recent queries
V1.1	Autocomplete, popular queries, synonyms, typo handling if supported
V2 - Search engine	Facets, typo tolerance, ranking rules, query analytics, fast large-catalog results
V2 - Personalization	Recently viewed, wishlist/cart affinity, category affinity, purchase exclusions
V3	Behavioral ranking/embeddings/ML only when data volume and business value justify it

Recommendation fallbacks

- No user data -> bestseller/category/popularity.
- Viewed product -> same category/brand/price band.
- Cart -> compatible or complementary products.
- Purchased -> exclude same durable item where appropriate; suggest accessories/replenishment.
- Always filter inactive/out-of-stock products according to UX policy.

Analytics events needed first

- home_section_impression/click
- search_submitted and zero_result_search
- product_impression/view
- variant_selected
- add_to_cart/remove_from_cart
- wishlist_add/remove
- checkout_started
- payment_method_selected
- payment_started/result
- purchase
- refund/return
- review_submitted

Notifications and Customer Communication

Separate transactional messages from marketing consent. Order updates should be reliable even when marketing is opted out.

Transactional events

Event	Channels
OTP / security	SMS primarily; email fallback where configured
Order placed/confirmed	In-app + SMS/email/push according to channel strategy
Payment success/failure	In-app + email/SMS/push
Packed/shipped/delivery exception/delivered	Push/SMS/in-app with tracking deep link
Return/refund updates	In-app + email/SMS/push
Support reply	In-app + push/email

Notification architecture

- Domain event -> queue -> channel provider -> delivery log.
- Template IDs and localized variables instead of hardcoded message strings in business logic.
- Deduplicate events and respect retry limits.
- Track provider message ID/status where available.
- Marketing messages require consent, unsubscribe and frequency controls.
- Deep links route directly to order, payment, product, campaign or support detail.

Analytics and Reports

Instrument the funnel from the start; build advanced dashboards only after event quality is trustworthy.

Business reports

Report	Dimensions / measures
Sales	GMV/net sales, orders, AOV by date/channel/category/brand/payment
Products	Units, revenue, conversion proxy, return rate, stockout days
Payments	Attempts, success rate, failure reason, gateway, refunds, reconciliation mismatch
COD	Placed, confirmed, shipped, delivered, returned, remittance outstanding
Customers	New vs returning, repeat purchase, cohort later
Promotions	Usage, discount cost, revenue, conversion lift where measured
Shipping	Courier SLA, delivery success, return-to-origin, zone performance
Search	Top queries, zero-result queries, search-to-product click/purchase

Data quality rules

- Define each KPI in writing.
- Use order/payment status filters consistently across reports.
- Backfill/repair events with versioning when tracking schema changes.
- Do not mix estimated frontend revenue with verified backend purchase revenue.
- Protect customer PII in analytics exports.

Security, Privacy and Operational Controls

E-commerce combines identity, money and PII. Security is a product requirement, not a deployment afterthought.

Minimum security controls

Area	Controls
Transport	HTTPS everywhere, HSTS when ready, secure TLS configuration.
Authentication	Strong hashing, OTP throttling, session revocation, MFA for admin/sensitive roles.
Authorization	RBAC server-side on every admin action; object-level access checks for customer orders/addresses.
API	Rate limiting, validation, bounded pagination, safe file upload, CSRF protection when cookie-based web sessions are used.
Payments	Server-side secrets, signed/validated webhooks, idempotency, amount/currency matching, redacted logs.
Data	Encryption at rest where applicable, least-privilege DB credentials, backups, retention rules.
Admin	Audit logs, sensitive-action confirmation, export permissions, session/device monitoring.
Infrastructure	Secret manager/env separation, firewall/WAF as appropriate, patching, dependency scanning.

Privacy/legal pages you should publish

- Terms of Service
- Privacy Policy
- Return/Refund Policy
- Delivery/Shipping Policy
- Payment/COD terms
- Warranty/authenticity policy where applicable
- Contact/customer care information
- Cookie/analytics consent where required by your implementation and legal advice

Compliance note

Payment provider onboarding, fees, settlement rules, tax/VAT, consumer rights and required business documents can change. Confirm current requirements with the provider and qualified local legal/accounting advice before launch.

Performance, SEO and Accessibility

Your first blueprint already called out responsive images, caching, lazy loading, accessibility and SEO. Now make them engineering gates.

Web performance

- Serve AVIF/WebP where supported; generate multiple responsive sizes.
- CDN-cache public catalog/media safely; use cache tags/versioning for invalidation.
- Lazy load below-the-fold sections and images; avoid loading 23 homepage modules at once.
- Prefetch only likely next navigation, not the whole catalog.
- Use skeletons with stable dimensions to reduce layout shift.
- Paginate product grids and admin tables; virtualize very large lists.
- Move email/SMS/report/search sync to queues.

SEO

- Render discoverable HTML for key storefront pages using Laravel/Blade (or your existing SSR approach), then hydrate/enhance interactive areas with Vue. Product/category/campaign pages should expose useful content and metadata without depending on a client-only render.
- Unique product/category slugs and metadata.
- Canonical URLs and noindex for low-value parameter combinations.
- Product/Offer/Breadcrumb structured data where accurate.
- XML sitemap(s), robots rules and redirect management.
- 404/410 strategy for removed products; recommend replacements.

Accessibility

- Semantic labels for controls/images
- Keyboard navigation and logical focus order on web
- Visible focus and sufficient contrast
- Scalable text and responsive layouts
- Form field labels, error summaries and accessible validation
- Do not rely on color alone for order/payment status

Testing Strategy

Commerce bugs are expensive. Test state transitions, calculations and retry behavior more heavily than visual snapshots.

Test pyramid

Layer	Focus
Unit/domain	Money math, promotion rules, stock/reservation, state machine transitions, eligibility.
Repository/API integration	DB transactions, idempotency, authorization, provider/courier adapters using sandbox/mocks.
Vue/Blade UI	Loading/error/empty/data, cart interactions, checkout validation, responsive dashboards.
End-to-end	Browse -> cart -> checkout -> COD; online sandbox payment; admin fulfillment; return/refund.
Resilience	Duplicate webhook, timeout, retry, network loss, stale cart, concurrent stock purchase.
Security	Access control, rate limits, injection/file upload, secret leakage checks.
Performance	Homepage/catalog/search load, image payload, API p95, admin export queues.

Payment test cases - must pass

- Successful payment
- Customer cancellation
- Provider failure
- Callback arrives twice
- Webhook arrives before browser redirect
- Redirect arrives but webhook delayed
- Network lost after customer pays
- Amount mismatch
- Expired order payment
- Partial refund
- Full refund
- Sandbox/live config separation
- Reconciliation discovers missed success

Deployment, Observability and Backups

A stable checkout needs operational visibility before real customers arrive.

Environments

Environment	Purpose
Local	Developer mocks/sandbox, seed data
Staging	Production-like DB schema, gateway/courier sandbox, full E2E testing
Production	Live credentials, hardened config, monitoring, backups

Observability

- Structured logs with request/order/payment IDs
- Error tracking for Laravel backend and Vue/JavaScript frontend
- API latency/error dashboards
- Queue depth/failure alerts
- Payment webhook failure and reconciliation alerts
- Low-stock and oversell alerts
- Courier integration failures
- Admin security/audit alerts
- Uptime checks for storefront, API and callbacks

Backup and recovery

- Automated DB backups + point-in-time recovery if supported
- Object storage versioning/backup for critical media/config exports
- Test restore regularly - a backup that has never been restored is only a hope
- Document RPO/RTO expectations
- Keep encryption keys/secrets outside backups where appropriate

REST API Blueprint

Names are illustrative; keep versioning and response/error conventions consistent across all modules.

Module	Example endpoints
Auth	POST /auth/register, /auth/login, /auth/otp/send, /auth/otp/verify, /auth/forgot-password, /auth/logout
Account	GET/PATCH /me; CRUD /me/addresses; GET /me/orders; /me/notifications; /me/sessions
Home	GET /home, /announcements, /collections/{slug}
Catalog	GET /categories, /brands, /products, /products/{slug}, /search/suggestions
Cart	GET /cart; POST/PATCH/DELETE /cart/items; POST/DELETE /cart/coupon
Checkout	POST /checkout/quote; POST /orders with idempotency key
Payments	POST /orders/{id}/payments; GET /payments/{id}; provider return/IPN/webhook routes
Orders	GET /orders/{id}; POST /orders/{id}/cancel; /return-requests
Shipping	GET /shipping/options; GET /shipments/{id}/tracking
Reviews	GET/POST /products/{id}/reviews; PATCH own review
Admin catalog	CRUD /admin/products/categories/brands/collections; imports/exports
Admin inventory	GET/PATCH stock via adjustment actions; stock movement history
Admin orders	Search/list/detail + action endpoints confirm/pack/ship/cancel/refund
Admin CMS	CRUD/reorder/publish /admin/home-sections, banners, announcements, pages
Admin finance	Payments, refunds, reconciliation, reports
Admin RBAC	Users/roles/permissions/audit logs/settings

Response/error conventions

- Stable error code + human message + optional field errors.
- Expose correlation/request ID for support.
- Use 409 for state conflicts/stock race where appropriate, 422 for validation, 401/403 correctly.
- Pagination metadata and sort/filter echo where useful.
- Never return stack traces or provider secrets to clients.

Admin Workflow Checklist by Department

Use this as the acceptance scope for each dashboard area.

Department	Daily workflow that must be possible
Catalog	Create product -> variants -> images -> price -> stock -> preview -> publish -> edit -> archive.
Marketing	Create campaign -> promotion -> collection/banner -> schedule homepage section -> preview -> publish -> review analytics.
Operations	Open new orders -> verify COD/paid -> pack -> label -> ship -> monitor exceptions -> complete/cancel.
Inventory	Receive/adjust stock -> inspect movement history -> low-stock list -> reconcile warehouse count.
Finance	Review online payments -> failed/mismatch queue -> issue refund -> reconcile settlement/COD remittance -> export report.
Support	Find customer/order -> view timeline -> reply -> initiate allowed cancel/return action -> document notes.
Management	View sales/orders/payment/shipping KPIs -> drill into exceptions -> export controlled reports.
Security/Admin	Manage staff roles -> review audit log -> rotate integration settings/secrets process -> disable compromised account.

12-Sprint Implementation Plan

Adjust sprint length to your team. The dependency order matters more than the calendar.

Sprint	Focus	Definition of outcome
Sprint 1	Foundation	Auth/RBAC skeleton, DB conventions, API errors, audit, environments, media storage.
Sprint 2	Catalog admin	Categories, brands, products, variants, images, price, publish/archive.
Sprint 3	Inventory + storefront catalog	Stock model, product listing/detail, filters, search V1.
Sprint 4	Dynamic landing CMS	Homepage section CRUD/reorder/schedule/preview; connect existing landing components.
Sprint 5	Customer auth/account	Phone/email flows, profile, addresses, sessions, wishlist.
Sprint 6	Cart	Guest/account cart, merge, reprice, coupon skeleton, warnings.
Sprint 7	Checkout + orders	Shipping quote, checkout validation, order snapshot/state machine, COD.
Sprint 8	Admin orders + fulfillment	Order queues/detail/actions, packing/shipping model, invoice.
Sprint 9	Online payment	Gateway sandbox, webhook/IPN, validation, retry, payment history, reconciliation baseline.
Sprint 10	Customer orders + returns	History, tracking, cancel, return/refund request, admin resolution.
Sprint 11	Courier + notifications	Courier adapter, tracking, SMS/email/push jobs, exception queues.
Sprint 12	Hardening + growth	Analytics events/reports, SEO/perf/a11y, security review, E2E, launch checklist, P1 promotions/reviews.

Parallel work

Design system, QA automation, infrastructure, analytics schema and provider onboarding can run in parallel, but do not let UI outrun the domain/API contracts.

Your Exact Next 14 Days

Since the landing page is already complete, this is the most practical immediate sequence.

Day	Work	Output
1	Write business rules	Single/multi-vendor decision, return/COD/shipping/stock/payment policies, status definitions.
2	Finalize schema	Core tables/migrations for identity, catalog, inventory, cart, order, payment, CMS, audit.
3	API conventions + auth/RBAC	Response/error format, tokens/sessions, admin role scaffolding.
4	Category/brand admin	CRUD, validation, slugs, sort order, media.
5	Product/variant admin	Create/edit/publish product, variants, images, price.
6	Inventory	Warehouse/default stock, adjustments, availability calculation.
7	Storefront catalog	Category/product APIs + product listing/detail UI.
8	Search/filter V1	Price/brand/category/attribute filters + sort + pagination.
9	Home CMS model	Section schema, banners/collections, schedule/visibility.
10	Connect your landing page	Replace hardcoded sections with GET /home typed payload.
11	Customer auth/profile	Phone/email login/register/recovery + address book.
12	Cart	Guest/account cart, merge, quantity and revalidation.
13	Checkout quote	Address + shipping + coupon + final totals; no payment yet.
14	Order + COD happy path	Idempotent order creation -> admin order detail -> confirm/pack/ship simulated -> customer order detail.

After Day 14

Only after the COD happy path is fully testable end to end should you connect the first online payment gateway. That keeps payment debugging separate from unfinished order/stock logic.

32 IMPLEMENTATION SECTION

Advanced Features - Add After Core Stability

These are valuable, but each introduces new accounting, data or operational complexity.

Feature	When to add	Dependencies
Verified reviews + Q&A	After delivered-order data is stable	Order items, moderation, media
Loyalty points	After refund accounting is solid	Ledger model; earn/reverse rules
Gift cards/store credit	After payment/refund ledger is solid	Financial ledger, expiry/liability rules
Wallet	Only with clear business/legal/accounting model	Double-entry-like ledger, refunds, security
Referral program	After user identity/attribution is reliable	Referral codes, fraud controls
Abandoned cart recovery	After consent + cart events	Messaging provider, frequency caps
Recommendations	After enough clean behavior data	Analytics events/search ranking
Price-drop/back-in-stock alerts	After inventory/price events	Notification preferences
Subscriptions	Only for repeatable products/business model	Recurring billing/provider support, renewal orders
Multi-warehouse allocation	When volume/geography needs it	Stock allocation, shipping optimization
Marketplace/seller dashboard	Only if business is truly multi-vendor	Seller onboarding, commission, payouts, seller inventory/orders/returns
B2B/wholesale	When needed	Tiered pricing, minimum quantities, tax/invoice rules, account approval

If you become a marketplace

Do not simply add seller_id to products. You will need seller onboarding/KYC, store profile, listing approval, commission calculation, seller order allocation, payout/settlement ledger, disputes, seller returns, seller performance, seller-specific shipping and a dedicated Seller Dashboard. Treat it as a major product phase.

Production Launch Checklist

Do not launch because the pages look finished. Launch when the system survives real failure modes.

Area	Launch gate
Catalog	All live products have valid price, image, category, variant SKU and stock policy.
Landing	All claims/campaigns/discounts/official badges come from real data; schedule/expiry tested.
Auth	OTP/login/recovery rate limits and session revocation tested.
Cart	Price/stock change warnings and guest-account merge tested.
Checkout	Server totals cannot be tampered; unsupported address/payment rules tested.
Orders	Double-submit does not duplicate; status transition guards and history work.
Payments	Sandbox E2E + callback/IPN + validation + duplicate events + mismatch + refund + reconciliation tested.
COD	Eligibility, confirmation, courier collection/remittance and returns are operationally defined.
Shipping	Fees/ETA/zone rules and courier exception path tested.
Returns	Eligibility, approval, receipt, restock and refund actions tested.
Admin	RBAC, audit logs, exports and destructive action confirmations tested.
Security	Secrets, HTTPS, permissions, rate limits, uploads, backups and admin MFA policy reviewed.
Performance	Homepage/product/search payload and image sizes meet targets; queues/cache/CDN configured.
Analytics	Purchase uses verified backend order/payment data; funnels and zero-result search tracked.
Operations	Support contact, policies, incident owners, payment/courier escalation and manual recovery runbook exist.
Backup	Restore test completed and documented.

Definition of Done for Every Feature

Use the same quality bar for catalog, checkout, dashboards and integrations.

- Domain rules documented and enforced on backend.
- Happy path + loading + empty + validation + permission + retry + failure paths implemented.
- API contract versioned/typed and frontend error states mapped clearly.
- RBAC and object-level authorization covered.
- Audit/logging added for sensitive admin and financial actions.
- Analytics events added where the feature affects funnel/business reporting.
- Unit/integration/UI/E2E tests added according to risk.
- Responsive light/dark/theme-aware UI; accessibility reviewed.
- Performance and pagination considered; no unbounded lists/queries.
- Documentation/runbook updated for operations if humans need to manage it.

The core goal

A premium e-commerce product is not a long list of screens. It is a reliable loop where merchandising creates demand, checkout converts it, operations fulfill it, finance reconciles it, customers can recover from problems, and admin can change the business without a new app release.

References and Provider Notes

Provider API versions, commercial terms and onboarding requirements can change. Always use the merchant documentation issued for your account when going live.

Source blueprint reviewed: Bangladesh-Focused Professional E-Commerce Landing Page Blueprint, including the 23-section homepage flow, CMS-readiness recommendation, payment/delivery checklist and trust rules. The original blueprint mentioned Laravel/Vue/Web as an implementation reference; this revised continuation targets the user's actual Laravel + Vue.js mixed website.

Current payment-provider references checked for this roadmap

SSLCOMMERZ Developer Documentation - https://developer.sslcommerz.com/doc/v4/
Documents session creation, IPN, validation, sandbox and hosted/easy checkout flows.

SSLCOMMERZ Sandbox Registration - https://developer.sslcommerz.com/registration/
Sandbox account and testing entry point.

shurjoPay REST API Documentation - https://web2.shurjopay.com.bd/frontend-developers/direct-integration-with-api
Documents authentication token, initiate/verify APIs and sandbox/live environments.

shurjoPay Integration Guide - https://web2.shurjopay.com.bd/frontend-developers/integration-howto
Sandbox and SDK/plugin integration guidance.

bKash Tokenized Checkout Terms - https://www.bkash.com/en/page/tokenized_checkout
Confirms bKash PGW tokenized checkout product terms; use current merchant/developer docs for API details.

Nagad Merchant Onboarding - https://nagad.com.bd/en/forms/?form=me
Public merchant onboarding entry point; use provider-issued integration docs for exact technical flow.

Final implementation principle

Build the commerce engine in layers, make the completed homepage a CMS client of that engine, and add payment/courier integrations as replaceable adapters. This sequence gives you a dynamic Bangladesh-ready store without coupling every business decision to the UI or to one gateway.